ISSN 0361-7688, Programming and Computer Software, 2025, Vol. 51, No. 8, pp. 587–604. © Pleiades Publishing, Ltd., 2025.

State of the Art of the Security of Code Generated by LLMs: A Multivocal Literature Review

Leonardo Criollo Ramírez[a] (ORCID: 0009-0000-0682-9854), Xavier Limón[a,*] (ORCID: 0000-0003-4654-636X), Ángel J. Sánchez-García[a,] (ORCID: 0000-0002-2917-2960), and Juan Carlos Pérez-Arriaga[a,b,***] (ORCID: 0000-0003-2354-2462)**

[a] *Facultad de Estadística e Informática, Universidad Veracruzana, Veracruz, México*

[b] *Universidad de Alcalá, Madrid, España*

**e-mail: hlimon@uv.mx*

***e-mail: angesanchez@uv.mx*

****e-mail: juaperez@uv.mx, juancarlos.pereza@edu.uah.es*

Received August 17, 2025; revised August 27, 2025; accepted August 27, 2025

Abstract—AI-assisted programming has gained significant traction in recent years, primarily driven by advances in large language model (LLM) technologies. This growth has resulted in the development of tools such as ChatGPT and GitHub Copilot. However, the use of AI models for code generation presents a critical challenge: The generated code is prone to security vulnerabilities, raising concerns within the domain of secure software development. This study examines the current research landscape on the security implications of LLM-generated code from a software engineering perspective. To this end, we conducted a multivocal literature review (MLR) in accordance with the guidelines outlined by Kitchenham et al. and Garousi et al. The search encompassed five academic sources: IEEE Xplore, ACM, ScienceDirect, Springer Link, and Wiley Online Library; as well as two grey literature sources: Google Scholar and Google. We supplemented the selection by means of backward and forward snowballing techniques. Through Quasi-Gold's automated search, we identified 2796 peer-reviewed studies, from which we selected 48 primary studies. The analysis revealed seven distinct categories of security vulnerabilities in LLM-generated code, six mitigation strategies and best practices, and four tools recommended for improving security in AI-assisted coding. While research on this issue remains in its early stages, initial findings highlight the need for caution when integrating AI-powered code generation into software development, as it may introduce vulnerabilities at a higher rate than human-written code. The vulnerabilities, mitigation strategies, and tools identified in this study can serve as valuable resources for developers seeking to use LLM-based programming assistants more securely and responsibly.

Keywords: LLM, code recommenders, secure software development, systematic literature review

DOI: 10.1134/S0361768825700446

1. INTRODUCTION

This study is an extension of our previous research presented at the 12th International Conference on Software Engineering Research and Innovation (CONISOFT) in 2024 [1]. Our initial work focused on a systematic literature review (SLR) delving into white literature. However, this updated iteration expands on the existing data by including insights from gray literature through the adoption of a multivocal literature review (MLR) approach. Furthermore, we have extended the research timeline to encompass findings up to March 2025. By incorporating additional sources and broadening our research scope, we have augmented our findings from the original 15 primary studies to a set of 48 primary studies, marking a substantive increase in our analyzed material. In this revised version, we have also expanded our methodological presentation and deepened our discussions to accommodate the new findings.

The field of software engineering is characterized by continuous advancement, driven by the emergence of novel technologies that enhance developer productivity and software quality. Among these innovations, coding assistants have been introduced to facilitate the software development process by automating the generation of repetitive code segments. While coding assistants are not a recent concept, the integration of artificial intelligence (AI) language models has substantially increased their capabilities and adoption. AI encompasses systems that exhibit intelligent behav-

ior by analyzing their environment and executing actions to achieve predefined objectives [2].

The incorporation of AI into coding assistants has led to the proliferation of sophisticated and widely utilized tools within the software development domain. One prominent category of such AI-based systems is large language models (LLMs), which are trained on extensive corpus of human-generated textual data. This training enables LLMs to process and produce human-like text, including programming code.

Advancements in LLMs have facilitated the development of tools capable of dynamically generating compilable code, assisting developers throughout the software creation process. A notable example is GitHub Copilot, which has attracted over a million users [3]. However, it is important to recognize that these models are trained on publicly available code repositories to enable their code generation capabilities [4]. Consequently, the training data utilized in these models may contain security vulnerabilities that could inadvertently or intentionally manifest in generated code, raising concerns about potential risks such as accidental or targeted data poisoning [5]. Additionally, many developers have reported hesitancy in adopting these tools or rejecting their generated code due to concerns regarding code quality [6].

The security implications of LLM-generated code have been highlighted in prior research. For example, Pearce et al. [7] found that prominent LLM-based solutions could generate code containing security vulnerabilities in up to 40% of cases. Similarly, Siddiq et al. [8] identified 44 distinct security-related code smells within the training datasets of the open-source LLM GPT-Neo. These findings underscore the necessity of assessing the security risks associated with LLM-powered coding assistants.

To address these concerns, this study employs a multivocal literature review (MLR) approach, as proposed by Garousi et al. [9] to synthesize findings from both academic research and grey literature regarding the security aspects of code generated by LLMs. This review examines the types and prevalence of vulnerabilities that may arise in LLM-generated code, potential attack vectors associated with these vulnerabilities, and the existing tools and techniques available for mitigating security risks when utilizing AI-driven code generation. Through this analysis, the study aims to provide a comprehensive overview of the security challenges associated with LLM-generated code and equip developers with the necessary insights to mitigate potential threats.

The remainder of this paper is structured as follows: Section 2 presents related work relevant to this study. Section 3 details the research methodology. Section 4 discusses the results and findings. Section 5 provides an in-depth discussion of the findings. Section 6 examines potential threats to validity associated with this research. Finally, Section 7 presents the conclusions.

2. RELATED WORK

Existing surveys have examined the capabilities of LLMs in code generation [10, 11]. These general studies provide multifaceted evaluations of LLM-generated code, addressing metrics such as performance, functional correctness, and, to a lesser extent, security. While these surveys offer valuable insights into the technical and practical aspects of code generation, they only briefly touch upon the security implications, often treating them as a secondary concern.

In contrast, security-focused reviews such as that by Yao et al. [12] provide broader analyses of LLMs within security contexts. However, their focus is not limited to code generation but encompasses a wide array of topics, including vulnerability detection in human-written code, automated test case generation, the synthesis of cyber-attacks, and the inherent vulnerabilities of LLM architectures themselves. As such, while security is treated in depth, the implications of LLM-generated code on software vulnerabilities are not their central focus.

More directly relevant to this study is the survey by Horne [13], which investigates the security concerns associated with LLM-powered programming assistants. This work systematically categorizes known vulnerabilities introduced during various stages of LLM deployment – from pre-training and fine-tuning to inference and downstream integration. The survey also compiles concrete findings from multiple empirical studies and proposes mitigation strategies aimed at both end-users and developers of these systems. One key recommendation is the separation of models used for code generation and vulnerability detection, thereby reducing the likelihood of undetected security flaws in automatically generated code.

The study by Kaniewski et al. [14] recounts multiple approaches other researchers have devised for vulnerability detection, localization and repair for AI generated code as alternatives to manual code review, which can be arduous and time consuming. The techniques the authors explore include using LLMs to evaluate the generated code, static and dynamic code analysis, automatic code patching, among others.

Unlike the two previous works mentioned, the present study emphasizes the perspective of software developers who interact with LLMs as programming assistants, rather than that of AI researchers who build and train these models. This distinction enables a

more practitioner-oriented discussion focused on actionable risks and mitigation strategies relevant to developers integrating LLMs into their workflows.

The study by Tony et al. [15] investigates several prompting techniques usable for secure code generation with LLMs, including zero-shot, one-shot, few-shot, among others. Additionally, the study includes an evaluation of each prompting technique and it's performance in validity and security benchmarks.

Finally, it is important to recognize the rapid and ever-changing nature of this research domain. Regular publication of new discoveries, particularly those shared in grey literature, emphasizes the necessity of continual review within the field to stay abreast of emerging challenges and best practices. It is worth noting that, unlike the studies mentioned earlier, our research also integrates findings from grey literature, providing a broader perspective that encompasses insights from the industry.

3. RESEARCH METHOD

This section details the methodological approach employed for conducting our multivocal literature review (MLR), following the guidelines proposed by Kitchenham et al. [16] and Garousi et al. [9] synthesizing both academic and grey literature in Software Engineering. We designed our MLR approach to capture a comprehensive and practice-informed view of the research questions by integrating diverse sources of evidence, including peer-reviewed publications and non-academic sources such as white papers, blog posts, technical reports, and practitioner forums, covering a wider view of the research subject.

To support the identification of relevant primary studies, we utilized the quasi-gold standard automated search technique developed by Zhang et al. [17], which is specifically tailored for systematic and reproducible literature identification in Software Engineering. We synthesized the results from the selected sources using a narrative synthesis approach, based on the framework outlined by Popay et al. [18], which is suitable for integrating findings from heterogeneous study types.

We conducted the MLR in multiple iterative stages, adapted from Garousi et al. [9], including the following key phases:

(1) Planning the review: Definition of research goals, scope, and inclusion of academic and grey literature sources.

(2) Review execution: Search, selection, and data extraction across both academic and practitioner-oriented content.

(3) Reporting and documentation: Synthesis of findings and critical analysis to answer the research questions.

We carried out the identification and selection of primary studies using a structured process comprising the following steps:

(1) formulation of research questions;

(2) definition of inclusion and exclusion criteria for both academic and grey literature;

(3) development and validation of search strategies;

(4) selection of data sources, including digital libraries (e.g., IEEE Xplore, ACM DL), search engines (e.g., Google Scholar), and practitioner platforms (e.g., Medium, Stack Overflow);

(5) execution of automated searches using the Quasi-Gold Standard workflow;

(6) relevance assessment and selection of primary studies;

(7) data extraction guided by a predefined schema.

To evaluate the effectiveness of the automated search component, we employed *Recall* and *Precision* metrics using the following formulas:

$$Recall = \frac{R_{\text{found}}}{R_{\text{total}}}, \quad (1)$$

$$Precision = \frac{R_{\text{found}}}{N_{\text{total}}}. \quad (2)$$

where:

- R_{total} is the number of relevant studies manually selected.
- N_{total} is the number of studies returned by the search string.
- R_{found} is the number of relevant studies returned by the search string.

The search workflow is illustrated in Fig. 1 [17].

3.1. Research Questions

To guide the review, we formulated four research questions (RQs) that reflect key concerns in secure software development practices involving LLMs. These questions are the basis of all MLR stages, including search, selection, extraction, and synthesis, as listed in Table 1.

3.2. Search Terms

In order to facilitate the creation of the search string, we compiled a list of terms related to the subject of the study. We categorized these terms into two sections: *LLMs* and *Secure Coding*. Table 2 shows the list of selected terms.

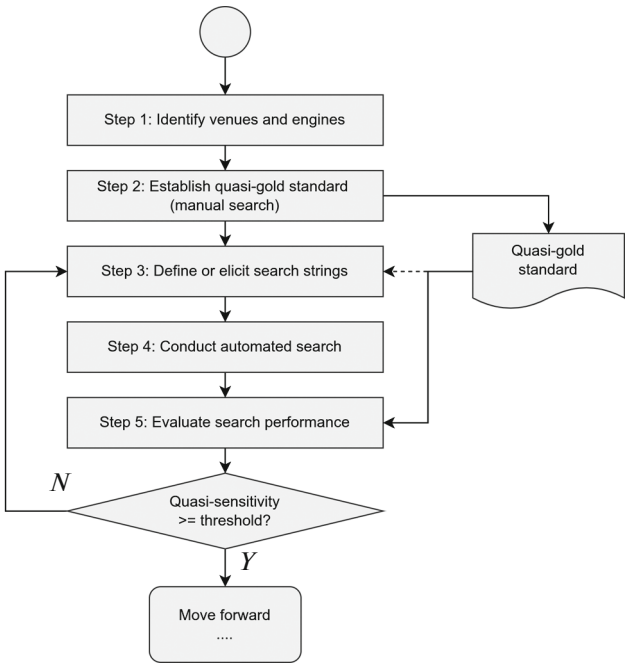

Fig. 1. Quasi-gold standard automated search workflow [17].

3.3. Search Strings

Following the guidance of Zhang et al. [17], we iteratively constructed and validated search strings to maximize recall while retaining precision. We used an initial pilot search on IEEE Xplore to calibrate performance metrics. The final query yielding the best results is the string corresponding to id 4 in Table 3:

```
((“Large Language Model” OR “Large Language Models” OR LLMs OR “Language Model” OR “Language Models” OR “Code Generation” OR “Code Contribution” OR “Coding Assistant”) AND (Security OR Vulnerability OR Vulnerabilities)).
```

3.4. Data Sources

We queried a range of peer-reviewed databases including IEEE Xplore, ACM Digital Library, SpringerLink, and Scopus. We based this choice on their relevance to software engineering research and the rigorous peer-review processes. Table 4 presents the selected digital libraries.

3.5. Grey Literature Exploration

Following MLR principles, we included grey literature to capture perspectives from industry practitioners. We searched Google Scholar using tailored queries and applied date-range filters from 2020 onward to align with the emergence of LLM-based code generation (e.g., GitHub Copilot).

The following criteria guided our inclusion of grey literature:

- relevance to the research questions;
- credibility of the source (e.g., author expertise, organizational affiliation);
- depth of technical discussion;
- publication date (2020 or later).

To assess findings quality, we adapted the checklist provided by Garousi et al. [9] for grey literature evaluation. This checklist includes dimensions such as source authority, objectivity, methodological transparency, relevance to the research subject, and literature level.

In this study, our exploration of grey literature was restricted to the first two levels of grey literature, as introduced by [9]. The 1st level encompasses high control and credibility sources: books, magazines, theses, governmental reports. On the other hand, the 2nd level includes mid-level control and credibility sources: annual reports, news articles, presentations, videos, Q&A sites, wiki articles, and official company blogs).

In order to assess the quality of each grey literature source, we employed the format proposed by Garousi, adapted to the specific needs of our research. This format can be seen in Table 5. For each question answered with “Yes”, the source would be awarded 1 point. At the end of the evaluation, all the points were averaged, and the source would be deemed trustworthy if the average score was equal or greater than 0.8.

Additionally, we employed retention criteria as proposed by Garousi in order to minimize the effort spent on irrelevant results. The criteria we utilized are as follows:

- theoretical saturation: New results not related to the subject of study;

– after 10 consecutive results not relevant to the subject of study, the search is stopped;

- limited effort: Only the first N search results are considered;

– only the first 150 search results are considered;

- evidence depletion: Extract all theoretical evidence from the source.

3.6. Study Selection Process

This section describes the inclusion and exclusion criteria employed for study selection, as well as the structured process used to apply these criteria throughout the MLR.

To ensure a systematic and replicable selection process, we applied a set of well-defined inclusion and exclusion criteria during the identification of relevant studies. We applied these criteria as progressive filters in order to incrementally narrow down the set of con-

sidered studies to those most aligned with our research objectives.

(1) Inclusion criteria (ICs). To focus on contemporary and relevant developments, we only considered studies published from 2020 onwards for inclusion. This temporal constraint reflects the emergence of LLM-based code generation technologies—such as OpenAI's GPT-3—which became publicly accessible in mid-2020. We excluded prior studies on the premise that they are less likely to offer insights directly applicable to current LLM capabilities. Table 6 summarizes the inclusion criteria employed.

(2) Exclusion criteria (ECs). We excluded studies that did not satisfy the inclusion criteria from the corpus. Common grounds for exclusion included outdated publication dates, lack of relevance to LLM-generated code, or lack of methodological rigor. Table 7 outlines the exclusion criteria applied.

3.7. Selection Process

We applied the selection criteria through a multistage filtering process. At each stage, we progressively introduced more specific filters to efficiently eliminate nonrelevant studies and reduce unnecessary review effort. Figure 2 illustrates the sequential stages of the selection procedure.

3.8. Snowballing

To augment the automated search and ensure comprehensive coverage of the literature, we employed both backward and forward snowballing, following the methodology described by Wohlin [19].

Backward snowballing involved examining the reference lists of initially selected primary studies to identify additional relevant literature. Forward snowballing, in contrast, entailed reviewing studies that cited the initial set of primary studies. We conducted these activities supported by academic tools such as Google Scholar and Connected Papers, which facilitate the automation of citation tracking.

New studies identified through snowballing underwent the same selection procedure as illustrated in Fig. 2, ensuring consistency and methodological rigor.

3.9. Data Extraction

We performed data extraction using a structured template designed to facilitate the systematic collection and organization of relevant information from each study. This template enabled consistent documentation and simplified later stages of analysis. The data extracted from each source included:

- title;
- authors;
- year of publication;
- source and publication type (e.g., journal, conference, grey literature);
- publication details (e.g., volume, issue, DOI);
- abstract;

Table 1. Research questions and their motivations

Identifier	Research questions	Motivation
RQ1	What vulnerabilities are related to code generated by LLM powered assistants?	To examine the types and prevalence of vulnerabilities in code produced by LLMs, thereby clarifying the security concerns associated with their adoption.
RQ2	What attacks are related to vulnerabilities found in code generated by LLM assistants?	To understand the attack vectors linked to these vulnerabilities, in order to assess the practical security risks of relying on LLM-generated code.
RQ3	What strategies and practices can be applied to mitigate the vulnerabilities of LLM generated code?	To compile and present actionable practices that developers can adopt to safely integrate LLM-based tools into their development workflow.
RQ4	What tools can be used to prevent and mitigate security vulnerabilities when using LLM generated code?	To identify relevant security tools that can support developers in reducing the risk of using code generated by LLM assistants.

Table 2. Search terms used to build the search string

Concept	Related terms
LLMs	Large Language Model, Large Language Models, LLMs, Language Model, Language Models, GPT-3, GPT-4, LlaMA, Programming Helper, Coding Assistant, Transformer, Code Generation, Code Contribution, ChatGPT, OpenAI Codex, Github Copilot
Secure Coding	Secure Coding, Security, Code Security, Vulnerabilities, Vulnerability, Threat, Threats, Weakness, Weaknesses, Bug, Bugs

Table 3. Proposed search strings

Id	Search strings	Studies found	Relevant studies found	Recall	Precision
1	((LLM OR "Large Language Model") AND (Security))	12	1	1/5 = 0.2	1/12 = 0.083
2	(("Large Language Model" OR "Large Language Models" OR LLMs OR "Language Model" OR "Language Models") AND (Security OR Vulnerability OR Vulnerabilities OR Threat OR Threats))	214	4	4/5 = 0.8	4/214 = 0.018
3	(("Large Language Model" OR "Large Language Models" OR LLMs OR "Language Model" OR "Language Models") AND (Security OR Vulnerability OR Vulnerabilities OR Risk OR Risks))	241	4	4/5 = 0.8	4/241 = 0.016
4	(("Large Language Model" OR "Large Language Models" OR LLMs OR "Language Model" OR "Language Models" OR "Code Generation" OR "Code Contribution" OR "Coding Assistant") AND (Security OR Vulnerability OR Vulnerabilities))	266	5	5/5 = 1	5/266 = 0.018
5	("Large Language Model" OR "Large Language Models" OR LLMs OR "Language Model" OR "Language Models" OR GPT OR GPT-3 OR GPT-4 OR LLaMA OR "Programming Helper" OR "Coding Assistant" OR Transformer OR "Code Generation" OR "Code Contribution" OR "ChatGPT" OR "OpenAI Codex" OR "Github Copilot") AND ("Secure Coding" OR "Code Security" OR Vulnerabilities OR Vulnerability OR Threat OR Threats OR Weakness OR Weaknesses OR Bug OR Bugs)	548	3	3/5 = 0.6	3/548 = 0.005

- corresponding research questions;
  - extracted data relevant to RQ1;
  - extracted data relevant to RQ2;
  - extracted data relevant to RQ3;
  - extracted data relevant to RQ4;
- general observations and contextual notes.

3.9. Data Extraction

We conducted the synthesis of extracted data following the narrative synthesis method, as proposed by Popay et al. [18]. This approach is well-suited for integrating qualitative findings from heterogeneous sources, including both academic and grey literature, as recommended for MLRs.

Table 4. Search terms used to build the search string

Search engine	URL
IEEE Xplore	https://ieeexplore.ieee.org/Xplore/home.jsp
ACM Digital Library	https://dl.acm.org/
Science Direct	www.sciencedirect.com/
Springer Link	https://link.springer.com/
Wiley Online Library	https://onlinelibrary.wiley.com/

We applied the narrative synthesis approach through the following stages:

- developing a theory for how, why and for who the synthesis is needed;
- developing a preliminary synthesis with findings of the primary studies;
- exploring relationships in the data;
- evaluating the synthesis robustness.

To support these stages, we generated several artifacts that enhanced transparency and traceability in the synthesis process.

(1) Textual descriptions of primary studies. We represented each included source through a detailed textual description, which served as a compact yet informative summary of the study's contribution to the research questions. We created these descriptions using the structured template mentioned in the data extraction phase. Each summary included a concise paraphrasing of key insights, methodological characteristics, and contextual information (e.g., domain, scope, and limitations). This artifact facilitated inter-

study comparison, which we used as the basis for subsequent thematic synthesis.

(2) Concept map of extracted findings. To visualize the thematic relationships and recurring patterns across the findings, we constructed a concept map. This map illustrates the connections among findings, concepts, and emergent categories identified during data synthesis. The construction process was iterative and grounded in both inductive coding and deductive alignment with the predefined research questions.

The concept map served two key purposes. First, it supported the identification of thematic clusters relevant to LLM-generated code security, such as common vulnerability patterns, tool-based detection methods, or mitigation strategies. Second, it enabled the identification of cross-cutting concerns that spanned both academic and grey literature sources, thus fulfilling a core requirement of multivocal literature reviews. This map can be found in Fig. 3.

3.9. Method Conduction

Upon executing the predefined search strategy using the most effective and validated search strings across the selected data sources, we compiled an initial set of candidate primary studies. We then processed this initial dataset through the multistage selection procedure described earlier.

The outcome of each phase of the selection process, including study inclusion and exclusion at each stage, is summarized in Table 8.

4. RESULTS

Following the completion of each phase of the selection procedure depicted in Fig. 2, we identified an initial set of 10 primary studies in academic literature. These studies served as the foundation for a subsequent round of forward and backward snowballing, conducted in accordance with the methodology described in Section 3.3. The snowballing process yielded additional potentially relevant studies, which we assessed using the same inclusion and exclusion criteria outlined previously. As a result, we incorporated five additional studies, bringing the total number of selected primary studies to 15 at that stage.

4.1. Extended Search and Additional Inclusions

To ensure that the review remained current with ongoing developments in the field, we extended the search cutoff date to March 2025. This extension enabled the identification of 20 additional peer-reviewed publications that met the established selection criteria.

Table 5. Quality assessment format

Criterion	Questions
Author's authority	• Is the source reputable? • Is the author associated with a reputable organization? • Has the author published other works in the field of security?
Methodology	• Does the author have a clear objective? • Does the source rely on contemporary references? • Does the work refer to a specific population or case?
Objectivity	• Are the author's claims supported by verifiable sources or are they opinions? • Is the author biased due to any conflict of interest? • Are the conclusions supported by data?
Date	• Does the study have an established date?
Position on related sources	• Have formal or gray literature sources been consulted or discussed?
Type of publication	• 1st level (measure=1): High control of output/High credibility: Books, journals, theses, government reports, white papers. • 2nd level (measure=0.5): Moderate media control/Moderate credibility: Annual reports, news articles, presentations, videos, Q&A websites (like StackOverflow), Wiki articles. • 3rd level (measure=0): Low media control/Low credibility: Blogs, emails, tweets.

Furthermore, the exploration of relevant grey literature resulted in the inclusion of 19 more studies. We rigorously evaluated each of these sources for credibility,

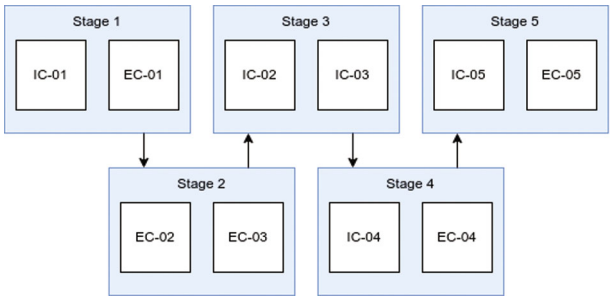

Fig. 2. Primary studies selection process.

Table 6. Inclusion criteria

ID	Criteria
IC-01	The study was published between 2020 and 2023
IC-02	The title or keywords of the study show indications of answering at least one research question
IC-03	If the area of knowledge can be identified, the study must belong to computer sciences.
IC-04	The abstract of the study shows indications of answering at least one research question.
IC-05	The study answers at least one research question.

Table 7. Exclusion criteria

ID	Criteria
EC-01	No access to full publication.
EC-02	Study is not written in English.
EC-03	Study is a technical report, book chapter, slideshow or poster.
EC-04	Duplicated study.
EC-05	Study based on a newer publication.

relevance, and methodological transparency, in line with the quality assessment principles recommended for multivocal literature reviews [9]. With these additions, the final corpus of selected primary studies consisted of a total of 48 studies, combining both academic and credible grey literature contributions.

Of the 48 primary studies included in this review, publication years are distributed as follows: one study appeared in 2021, four in 2022, sixteen in 2023, eighteen in 2024, and nine in 2025, despite it being early in the year, suggesting strong research interest, as illustrated in Fig. 4. This distribution highlights a clear upward trajectory in research on the subject, suggesting rapidly accelerating academic and industry interest in this topic over recent years.

Table 9 presents an overview of all selected primary studies, including those identified during the initial search phase and through subsequent snowballing iterations.

In the following section, we relate our findings to each of our research questions.

4.2. RQ1. What Vulnerabilities Are Related to Code Generated by LLM Powered Assistants?

The presence and frequency of vulnerabilities in code generated by LLMs vary depending on the programming context and the complexity of the task [29]. LLMs present in technologies such as GitHub Copilot and ChatGPT have been shown to produce vulnerable code more frequently than human developers working independently [7, 20, 26]. Studies have reported that 40 to 70% of generated code may contain vulnerabilities [7, 21], with some models generating faulty code at twice the rate of correct code [20].

We identified in the literature the following categories of vulnerabilities in LLM-generated code:

• **Simple, stupid bugs (SStuBs):** These are one-statement bugs that compromise code quality or functionality, and are often overlooked by static analysis tools. Codex, the model behind GitHub Copilot, produces SStuBs at twice the rate of correct code [20]. These bugs are simple to fix once identified, but may take an average of 240 days to resolve due to their subtlety and developers' reliance on generated code [62].

• **Common weakness enumerations (CWEs):** LLMs frequently replicate vulnerabilities from the MITRE Top 25 CWE list [5]. This is attributed to the prevalence of such flaws in training datasets sourced from open-source code [4, 42]. For instance, CodeQL analysis detected 248 CWE instances across 19 types in

Table 8. Number of primary studies per selection phase

Selection phase	Initial search	Stage 1	Stage 2	Stage 3	Stage 4	Stage 5
ACM Digital Library	1647	230	230	27	12	8
IEEE Xplore	1285	1285	1285	127	55	19
Science Direct	1161	142	131	5	0	0
Springer Link	845	839	172	23	5	0
Wiley Online Library	194	194	194	35	8	0
Forwards snowballing	18	13	7	5	5	5
Backwards snowballing	8	7	7	7	7	0
Google Scholar	9950	82	82	60	26	15
Google	10110	4	0	0	0	1
Total	25218	2796	2108	289	118	48

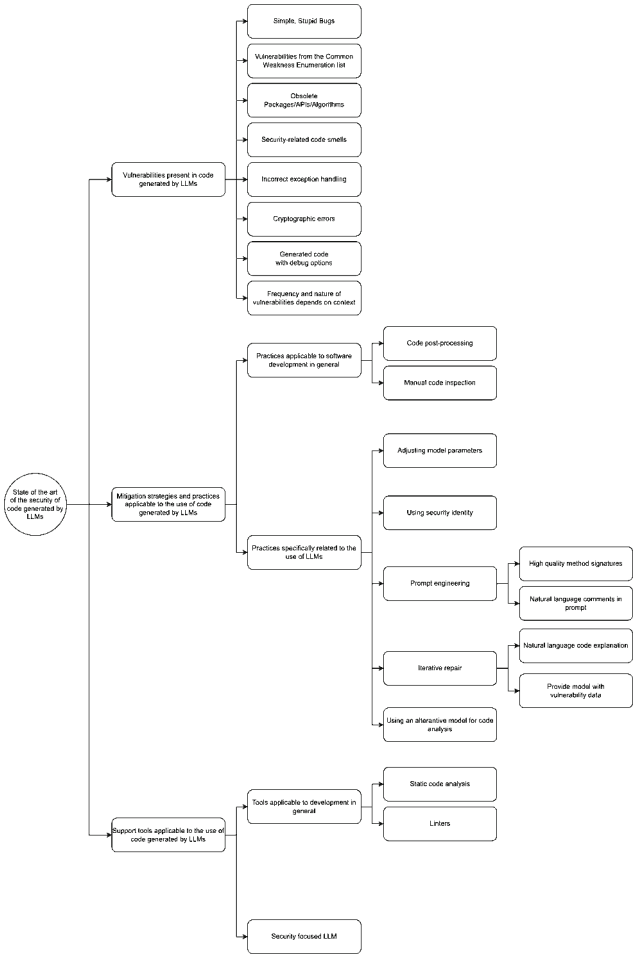

Fig. 3. Concept map. Own creation.

ChatGPT-generated code [40], with critical vulnerabilities such as CWE-215, CWE-476, CWE-330, CWE-78, CWE-89, CWE-676, CWE-20, CWE-120, CWE-125/787, CWE-190/191 and CWE-674 [34, 36, 43, 49, 52, 55, 59, 60]. We found that CWE-400 is the most common CWE in ChatGPT generated Python code compared to human-made code [45]. The ratio of code containing CWEs could reach as high as approximately 40% in C code generated with GPT-J [57] and 49% in Java, Python and C code generated by ChatGPT [53]. Furthermore, higher severity vulnerabilities were introduced more often than by humans [51]. All of this can be attributed to LLMs such as GPT-4, Claude 3 and Llama 3 failing to identify vulnerable code in anywhere from 60 to 88% of cases [50]. It should be noted that newer versions of the same LLM seem to produce reduced amounts of CWEs [46].

• **Outdated packages, APIs, and algorithms:** Due to training data containing older or poorly maintained projects, LLMs often generate code using deprecated libraries or insecure APIs [22, 23]. The security risks are compounded by the infrequent retraining of LLMs and the widespread misuse of APIs (up to 85% in GitHub repositories) [63].

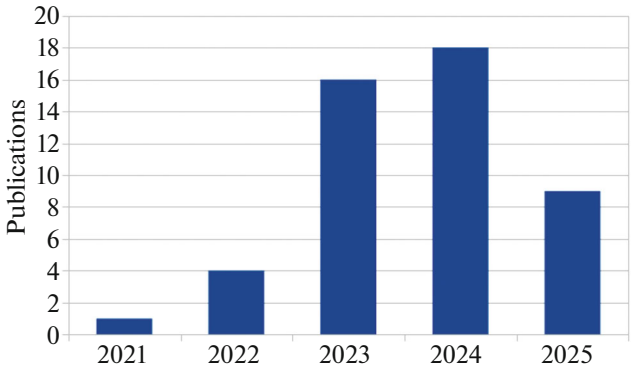

Fig. 4. Publication dates for selected primary studies.

• **Security-related code smells:** These refer to code patterns that suggest potential vulnerabilities. Tools like Pylint have identified 44 types of security-related code smells in training data and output of models such as GPT-Neo and Copilot, including B101 (use of assert) and B303 (use of insecure hash functions) [8].

• **Incorrect/incomplete exception handling:** LLMs show a high rate (up to 88.04%) of flawed exception handling [28]. Issues include incomplete coverage of potential exceptions, catching incorrect exception types, and overuse of try-catch blocks for non-exceptional flows.

• **Cryptographic errors:** Up to 70% of code involving cryptographic operations exhibits flawed API usage [33]. Common mistakes include predictable keys, insecure initialization vectors, improper certificate validation, and insecure storage of secrets.

• **Debug mode enabled in production:** Several LLMs, including GPT-3.5, GPT-4, and Bard, have been shown to produce Flask web apps with debug mode enabled by default [34]. This configuration exposes sensitive details and violates access controls, corresponding to high-severity CWEs such as CWE-215 and CWE-489.

Details of these findings can be found in Table 10.

4.3. RQ2. What Attacks Are Related to Vulnerabilities Found in Code Generated by LLM Assistants?

Despite an expanded review process incorporating both peer-reviewed and grey literature sources, we do not find studies that explicitly map real-world attacks or threat scenarios to the specific vulnerabilities observed in code generated by LLM-powered assistants. The lack of direct mappings suggests that there are currently no known attacks specifically targeting code generated by LLMs, as this subject is not directly addressed by the authors.

Table 9. Selected primary studies

Identifier	Title	Year	Reference
PS1	Large Language Models and Simple, Stupid Bugs	2023	[20]
PS2	Poisoning Programs by Un-Repairing Code: Security Concerns of AI-generated Code	2023	[5]
PS3	Efficient Avoidance of Vulnerabilities in Auto-completed Smart Contract Code Using Vulnerability-constrained Decoding	2023	[21]
PS4	The (ab)use of Open Source Code to Train Large Language Models	2023	[4]
PS5	AI-Driven Development Is Here: Should You Worry?	2022	[22]
PS6	LLMSecEval: A Dataset of Natural Language Prompts for Security Evaluations	2023	[23]
PS7	An Empirical Study of Code Smells in Transformer-based Code Generation Techniques	2022	[8]
PS8	Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions	2022	[7]
PS9	How Important Are Good Method Names in Neural Code Generation? A Model Robustness Perspective	2023	[24]
PS10	Grounded Copilot: How Programmers Interact with Code-Generating Models	2023	[25]
PS11	Do Users Write More Insecure Code with AI Assistants?	2023	[26]
PS12	Flexible and Secure Code Deployment in Federated Learning using Large Language Models: Prompt Engineering to Enhance Malicious Code Detection	2024	[27]
PS13	From Misuse to Mastery: Enhancing Code Generation with Knowledge-Driven AI Chaining	2023	[28]
PS14	PwnPilot: Reflections on Trusting Trust in the Age of Large Language Models and AI Code Assistants	2024	[13]
PS15	Refining ChatGPT-Generated Code: Characterizing and Mitigating Code Quality Issues	2024	[29]
PS16	Is GitHub's Copilot as Bad as Humans at Introducing Vulnerabilities in Code?	2022	[30]
PS17	Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants	2022	[31]
PS18	Evaluating Large Language Models Trained on Code	2021	[32]
PS19	An Investigation into Misuse of Java Security APIs by Large Language Models	2024	[33]
PS20	Occasionally Secure: A Comparative Analysis of Code Generation Assistants	2024	[34]
PS21	No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT	2023	[35]
PS22	Security Weaknesses of Copilot Generated Code in GitHub	2023	[36]
PS23	A User-centered Security Evaluation of Copilot	2024	[37]
PS24	Can ChatGPT replace StackOverflow? A Study on Robustness and Reliability of Large Language Model Code Generation	2023	[38]

Table 9. (Contd.)

Identifier	Title	Year	Reference
PS25	How Secure is Code Generated by ChatGPT?	2023	[39]
PS26	Just another copy and paste? Comparing the security vulnerabilities of ChatGPT generated code and StackOverflow answers	2024	[40]
PS27	Introducing Microsoft Security Copilot: Empowering defenders at the speed of AI	2023	[41]
PS28	Is Your AI-Generated Code Really Secure? Evaluating Large Language Models on Secure Code Generation with CodeSecEval	2024	[64]
PS29	Can We Trust Large Language Models Generated Code? A Framework for In-Context Learning, Security Patterns, and Code Evaluations Across Diverse LLMs	2024	[42]
PS30	Quality Assessment of ChatGPT Generated Code and their Use by Developers	2024	[43]
PS31	Fight Fire With Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?	2024	[44]
PS32	AI Writes, We Analyze: The ChatGPT Python Code Saga	2024	[45]
PS33	Assessing the Security of GitHub Copilot's Generated Code - A Targeted Replication Study	2024	[46]
PS34	Benchmarking Prompt Engineering Techniques for Secure Code Generation with GPT Models	2025	[47]
PS35	CWEval: Outcome-driven Evaluation on Functionality and Security of LLM Code Generation	2025	[48]
PS36	Security and Quality in LLM-Generated Code: A Multi-Language, Multi-Model Analysis	2025	[49]
PS37	Do LLMs Consider Security? An Empirical Study on Responses to Programming Questions	2025	[50]
PS38	Comparing Human and LLM Generated Code: The Jury is Still Out!	2025	[51]
PS39	Weaknesses in LLM-Generated Code for Embedded Systems Networking	2024	[52]
PS40	Navigating (in)Security of AI-Generated Code	2024	[53]
PS41	RefleXGen: The unexamined code is not worth using	2025	[54]
PS42	CodeLMSec Benchmark: Systematically Evaluating and Finding Security Vulnerabilities in Black-Box Code Language Models	2024	[55]
PS43	Balancing Security and Correctness in Code Generation: An Empirical Study on Commercial Large Language Models	2025	[56]
PS44	Fine-Tuning Large Language Model for Secure Code Generation	2024	[57]
PS45	PromSec: Prompt Optimization for Secure Generation of Functional Source Code with Large Language Models (LLMs)	2024	[58]
PS46	A Pilot Study on Secure Code Generation with ChatGPT for Web Applications	2024	[59]
PS47	Do Large Language Models Recognize Python Identifier Swaps in Their Generated Code?	2024	[60]
PS48	Securing Large Language Models: Threats, Vulnerabilities and Responsible Practices	2024	[61]

Table 10. Vulnerabilities present in LLM generated code

Type of vulnerability	Study
Simple, stupid bugs	[20]
CWEs	[4, 5, 7, 34, 36, 40, 42, 43, 46, 49–53, 55, 57, 59, 60]
Code smells	[8]
Smart contract specific vulnerabilities	[21]
Outdated APIs/packages/algorithms	[22, 23]
Incorrect/incomplete exception handling	[28]
Cryptographic errors	[33]
Debug mode enabled in production	[34]

While existing literature provides detailed accounts of common vulnerabilities in LLM-generated code, connections to concrete attack vectors remain unexplored. Future work may address this gap by systematically mapping known exploits and attack patterns—such as those catalogued in MITRE ATT&CK or similar frameworks—to the types of vulnerabilities uncovered in LLM-generated code. Such efforts could enhance our understanding of the practical implications of these weaknesses and support the development of more effective mitigation strategies.

4.4. RQ3. What Strategies and Practices Can Be Applied to Mitigate the Vulnerabilities of LLM-Generated Code?

The literature identifies a diverse set of mitigation strategies designed to reduce the prevalence of vulnerabilities in code generated by LLMs. These practices can be broadly categorized into general software development practices and techniques specifically tailored to the use of LLMs as code generation tools.

General development practices: Across several primary studies, postprocessing of generated code using static and dynamic analysis tools is commonly recommended to identify and mitigate security issues that may be introduced by both humans and LLMs [8, 21]. Manual code reviews are also emphasized as essential, particularly when integrating external code recommendations from LLMs. This process allows for the identification of subtle or context-specific security issues that automated tools may miss [28, 32, 36, 37, 40].

LLM-specific mitigation techniques: Prompt engineering is a central theme in LLM-specific mitigation strategies. Techniques include the use of descriptive natural language comments, which have been shown to influence model behavior and improve both the correctness and security of the output [20]. The inclusion of high-quality function signatures and carefully chosen method names significantly improves the alignment between prompts and the resulting code, thus reducing functional and security errors [24]. Literature studies explore the benefits and limitations of explicitly stating security goals in prompts. For instance, the use of simple instructions like "Your code should be secure and should NOT contain any vulnerability" led to measurable improvements in output security across multiple LLMs [48].

The use of iterative repair techniques is another prominent approach. These involve repeated evaluations and refinements of generated code, either manually or automatically. Techniques such as RefleXGen [54] leverage security knowledge bases to iteratively assess and revise output. Other studies report that vulnerability-aware iterative prompting can lead to substantial reductions in CWE instances and improvements in code quality, even outperforming more advanced models under certain conditions [47, 64]. Notably, techniques such as KPC (Knowledge-Prompt-Code) have shown effectiveness in guiding LLMs to adhere to best practices during code generation [28]. Assigning a "secure developr" persona to LLMs has also demonstrated potential in improving security posture, though effectiveness varies by model [34].

Additional strategies include the use of one-shot prompting with secure code exemplars, which reduces vulnerability incidence in single-generation scenarios [42]. Moreover, employing an alternate LLM to analyze or validate the output of the original generation process helps address weaknesses and potential model-specific backdoors [13]. Finally, proactive tuning of LLM parameters—particularly temperature—has been associated with significant improvements in output safety and consistency [26, 56].

We provide a summary of these strategies and their effectiveness in Table 11.

4.4. RQ3. What Strategies and Practices Can Be Applied to Mitigate the Vulnerabilities of LLM-Generated Code?

The literature identifies several tools that can aid in the prevention and mitigation of security vulnerabilities when integrating code generated by LLMs. These tools can be grouped into three main categories: static code analysis tools, linters, and security-focused language models.

Table 11. Vulnerability mitigation strategies and practices that can be applied to LLM code generation

Strategy or practice	Study
Code postprocessing	[8, 21]
Manual code inspection	[28, 32, 36, 37, 40]
Adjusting model parameters	[26, 56]
Using security identity	[34]
Prompt engineering	[20, 24, 48]
Iterative repair	[27–29, 42, 47, 54, 64]
Using an alternative model for analysis	[13]

Static code analysis tools: Static analyzers examine source code without executing it, enabling early identification of errors, code smells, and security vulnerabilities. Their use is widely recommended when incorporating LLM-generated code, as such code – despite being functional—often lacks security guarantees [8]. By detecting recurring vulnerability patterns and structural issues, these tools help developers enhance the robustness and trustworthiness of LLM-assisted code.

Several tools have been recommended across the studies reviewed, including CodeQL [34–36], [40, Bandit [8], Flake8 [13], and PMD [29]. The choice of tool typically depends on the programming language, the context of use, and the domain of the project.

Linters: Linters focus on improving the style, readability, and maintainability of code by enforcing coding conventions and detecting common formatting or stylistic issues. While they do not directly identify security flaws, their use complements static analyzers by ensuring that LLM-generated code adheres to project standards and best practices. This can reduce long-term maintenance overhead and prevent the introduction of hard-to-read or inconsistent code.

Studies recommend tools such as Pylint for Python [8] and Checkstyle for Java [29]. Given that LLMs can produce syntactically correct code that disregards style conventions, linters provide an important layer of quality control, particularly in collaborative and large-scale development environments.

Security-focused LLMs: Another emerging category includes LLMs that are explicitly trained or fine-tuned with security in mind. Since the behavior of generative models is heavily influenced by their training data, leveraging models designed for secure code generation can offer tangible improvements in the quality of code produced.

Security Copilot [41], developed by Microsoft, exemplifies this trend. It is tailored for cybersecurity analysts and trained to detect and reason about security-relevant aspects of codes. Its use is recommended in high-assurance environments where security is a top priority, particularly when paired with other tools like GitHub Copilot.

A summary of these tools is presented in Table 12.

5. DISCUSSION

Our MLR reveals that a significant proportion of code generated by LLMs, including ChatGPT, GitHub Copilot, GPT-Neo and others, generate code with vulnerabilities. These weaknesses pose substantial risks to software integrity, particularly when developers adopt generated code without critical evaluation. Empirical studies report that between 40% and 70% of LLM-generated code may contain vulnerabilities [7, 21], with some models producing incorrect or vulnerable code at a higher rate than secure alternatives [20].

This risk is especially pertinent for novice developers, who are among the primary users of AI-assisted coding tools [65], especially in learning environments where these kinds of tools are becoming more prevalent [66]. These users may incorrectly assume the correctness or security of the generated output, underscoring the importance of integrating secure development principles into educational and training initiatives focused on LLM-assisted programming.

Despite the proliferation of LLMs in software development workflows, no studies to date have introduced a formal threat model that explicitly incorporates LLMs as code-generating agents. This gap may be attributed to the context-sensitive nature of the vulnerabilities observed. Nevertheless, the vulnerabilities found in LLM-generated code often correspond to well-established categories such as those catalogued in the MITRE CWE Top 25 and OWASP Top 10 [5, 40]. This alignment suggests that flaws in the training data – such as the inclusion of outdated, insecure, or low-

Table 12. Tools that can be applied to LLM code generation

Tool suggested	Study
Static code analysis	[8, 13, 29, 34–36, 40]
Linters	[8, 13, 29]
Security focused LLM	[41]

quality code—are being propagated through model outputs.

Several recurring vulnerability patterns have been identified, including:

• High incidence of Common Weakness Enumeration vulnerablities (e.g., CWE-215, CWE-89, CWE-120) [34, 36].

• Improper cryptographic practices, with up to 70% of cryptographic code exhibiting flawed API use [33].

• Widespread issues with exception handling, particularly incomplete or incorrect handling logic [28].

• The use of outdated APIs or insecure dependencies due to outdated training data [23].

• Debug configurations left enabled in production code [34].

To mitigate these issues, the literature proposes both general software engineering practices and LLM-specific techniques. Standard practices such as static analysis and the use of linters (e.g., CodeQL, Bandit, Pylint) remain essential for identifying code smells and structural vulnerabilities [8, 40]. These tools are particularly useful given the frequency of subtle bugs, such as simple one-line logic errors (SStuBs), that escape manual review [20].

Mitigation strategies tailored to LLM-generated code generally fall into two categories: *prompt engineering* and *iterative refinement*. Prompt engineering includes the use of descriptive comments, clear method signatures, and explicit security goals to influence model behavior [24, 48]. Iterative refinement strategies, such as RefleXGen and Knowledge-Prompt-Code (KPC), employ repeated evaluations of the output using the same or auxiliary models to reduce the prevalence of vulnerability [28, 54]. In particular, alternate model validation and parameter tuning (e.g. reducing model temperature) have also shown promise in improving the quality and consistency of generated code [26, 47].

It is important to note that the applicability of these strategies can vary based on the mode of LLM interaction. Chatbot-based systems like ChatGPT offer greater control over prompts compared to in-IDE solutions such as GitHub Copilot, where prompt transparency and user influence may be limited [25].

Another thing worth mentioning is that as LLM tools evolve and are updated by their developers, they appear to be able to implement security standards with greater efficacy and are less prone to producing vulnerable code, even if they are still not perfect [46].

In the future, the development of standards and guidelines concerning the responsible use of LLMs in secure software development is critical. This includes establishing clear accountability frameworks for cases where insecure code is integrated into production systems due to overreliance on model output. In high-assurance or safety-critical domains, the use of LLM-based code generation tools may warrant restriction or additional layers of scrutiny.

6. THREATS TO VALIDITY

Ensuring the reliability and robustness of findings is a central concern in conducting a MLR. This section outlines potential threats to the validity of our study and describes the mitigation strategies employed to address them.

One limitation of the initial search process was restricted access to certain publications, which may have led to the inadvertent omission of relevant studies during the selection phase. Additionally, due to the emerging nature of research on LLM-generated code and the inherent difficulty of exhaustive manual search processes, we encountered a limited number of candidate studies suitable for evaluation using a quasi-gold standard approach.

To mitigate these limitations, we adopted a backward and forward snowballing strategy. This iterative method allowed us to identify additional relevant sources by examining the references and citations of initially selected studies. Snowballing proved effective in uncovering further contributions that we may have missed during the initial keyword-based search, thus enhancing the quality of our review.

Furthermore, we expanded the temporal and topical scope of our study by incorporating grey literature and extending the cutoff date of research up to March 2025. Including grey literature enabled us to integrate industry perspectives and insights from non-peer-reviewed sources, which are particularly valuable in fast-evolving fields where academic research may lag behind practical advancements.

These methodological adaptations aimed to increase the completeness and representativeness of our dataset, thereby improving the overall validity and relevance of our findings in the context of Software Engineering and secure development practices.

7. CONCLUSIONS

In this MLR, we examined the security of code generated by LLMs by reviewing a total of 48 distinct primary studies. Our aim was to identify common vulnerabilities, assess mitigation techniques, and explore future directions for secure LLM-enabled development. This review integrates both peer-reviewed and gray literature, enabling an integral understanding of the field. LLMs, trained on large-scale internet codebases, inherit not only the functional capabilities of programming languages but also potentially insecure patterns and flawed code. This raises significant concerns about the unintended introduction of vulnerabilities into software systems when relying on AI-powered code generation tools such as GitHub Copilot or ChatGPT. Our study reveals that LLM-generated

code is prone to a spectrum of vulnerabilities, including Simple Stupid Bugs, Common Weakness Enumeration (CWE) entries, security-related code smells, poor exception handling, insecure cryptography usage, use of deprecated libraries, and even domain-specific issues such as smart contract vulnerabilities. The studies highlight that the likelihood and nature of vulnerabilities depend on multiple factors, including the programming language used, task complexity, integration context, and prompt design.

Comparisons across studies show that LLM-generated code often exhibits a higher rate of vulnerabilities than human-written counterparts. These findings reinforce the need for critical evaluation and cautious integration of AI-generated code into production environments.

Although we do not found studies that explicitly link the vulnerabilities in LLM-generated code to real-world attacks, the consistency with known exploit patterns (e.g., those catalogued by MITRE and OWASP) highlights a latent threat. The absence of a formal threat model for LLMs underscores a need for future work to bridge this knowledge gap.

To counteract the risks identified, the literature proposes a number of mitigation strategies. These include prompt engineering techniques (e.g., including natural language comments or crafting detailed function signatures), model parameter tuning, iterative repair workflows, and postprocessing of generated code. These methods demonstrate that security can be influenced not only post hoc, but also proactively during the generation process. Some techniques, like security identity usage, appear to degrade performance in certain LLMs but not in others—highlighting the need for future empirical evaluation tailored to commonly used models like GPT-3.5, GPT-4, and Codex.

Complementary to these practices, we identified a growing ecosystem of tools that can support secure LLM usage, including security-specific static analyzers (e.g., CodeQL, Bandit), linters (e.g., Pylint, Checkstyle), and security-aware language models such as Microsoft's Security Copilot. The effective use of such tools depends on context, expertise, and integration into continuous development pipelines. We can also advise the usage of more traditional software bots to aid in developing tasks [67] to further improve results.

Synthesizing these findings, we conclude that a paradigm shift is needed in how software engineers approach the use of LLMs in code generation. The current assumption that AI-generated code is "good enough" for production use must be re-evaluated in light of empirical evidence to the contrary. Moreover, secure-by-default configurations and enhanced user education must become central pillars of future LLM tooling and deployment strategies. Furthermore, our review highlights the crucial role of organizational responsibility in fostering secure practices. Transparency, accountability, and the development of guidelines are essential for responsibly deploying LLMs in software development pipelines. The importance of education is also underscored; developers must be informed not only of the capabilities of LLMs but also of their limitations, particularly the risk of hallucinations [68] and contamination from insecure training data [4, 5].

In conclusion, while LLMs hold immense potential for enhancing productivity and innovation in software engineering, their use must be tempered with robust safeguards. As these tools become ubiquitous, empowering developers with the right knowledge and tools to use them responsibly is paramount. Future research should continue to explore more reliable evaluation methods, better secure-by-design LLM architectures, and comprehensive mitigation frameworks to ensure LLMs contribute to secure, trustworthy software development.

As a potential direction for future work, the categorized vulnerabilities, mitigation strategies, and recommended tools identified in this review could be integrated into a conceptual threat modeling framework tailored for LLM-assisted development. Such a model would help developers anticipate risks introduced by generated code and guide them in applying appropriate mitigation strategies. For example, vulnerabilities linked to common CWEs or outdated dependencies could be mapped to threat agents and attack vectors in developer-centric scenarios, enabling proactive security planning during design and code review phases. This type of framework could ultimately serve as a decision support tool in secure-by-design pipelines that incorporate LLM generated code.

FUNDING

This work was supported by ongoing institutional funding. No additional grants to carry out or direct this particular research were obtained.

CONFLICT OF INTEREST

The authors of this work declare that they have no conflicts of interest.

REFERENCES

1. Ramírez, L.C., Limón, X., Sánchez-García, Á.J., and Pérez-Arriaga, J.C., State of the art of the security of code generated by LLMs: A systematic literature review, *2024 12th International Conference in Software Engineering Research and Innovation (CONISOFT)*, Escondido, Mexico, 2024, IEEE, 2024, pp. 331–339. https://doi.org/10.1109/conisoft63288.2024.00050
2. Sheikh, H., Prins, C., and Schrijvers, E. (Research for Policy), Artificial intelligence: Definition and background, *Mission AI*, Cham: Springer, 2023, pp. 15–41. https://doi.org/10.1007/978-3-031-21448-6_2

3. Dastin, J., Microsoft attracting users to its code-writing, generative AI software, Reuters, 2023. https://www.reuters.com/technology/microsoft-attracting-users-its-code-writing-generative-ai-software-2023-01-25/.
4. Al-Kaswan, A. and Izadi, M., The (ab)use of open source code to train large language models, *2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering (NLBSE)*, Melbourne, 2023, IEEE, 2023, pp. 9–10. https://doi.org/10.1109/nlbse59153.2023.00008
5. Improta, C., Poisoning programs by un-repairing code: Security concerns of AI-generated code, *2023 IEEE 34th International Symposium on Software Reliability Engineering Workshops (ISSREW)*, Florence, 2023, IEEE, 2023, pp. 128–131. https://doi.org/10.1109/issrew60843.2023.00060
6. Liang, J.T., Yang, Ch., and Myers, B.A., A large-scale survey on the usability of AI programming assistants: Successes and challenges, *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, Lisbon, 2024, Association for Computing Machinery, 2024, p. 52. https://doi.org/10.1145/3597503.3608128
7. Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., and Karri, R., Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions, *2022 IEEE Symposium on Security and Privacy (SP)*, San Francisco, 2022, IEEE, 2022, pp. 754–768. https://doi.org/10.1109/sp46214.2022.9833571
8. Siddiq, M.L., Majumder, S.H., Mim, M.R., Jajodia, S., and Santos, J.C.S., An empirical study of code smells in transformer-based code generation techniques, *IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM)*, Limassol, Cyprus, 2022, IEEE, 2022, pp. 71–82. https://doi.org/10.1109/SCAM55253.2022.00014
9. Garousi, V., Felderer, M., and Mäntylä, M.V., Guidelines for including grey literature and conducting multivocal literature reviews in software engineering, *Inf. Software Technol.*, 2019, vol. 106, pp. 101–121. https://doi.org/10.1016/j.infsof.2018.09.006
10. Wang, J. and Chen, Yi., A review on code generation with LLMs: Application and evaluation, *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*, Beijing, 2023, IEEE, 2023, pp. 284–289. https://doi.org/10.1109/medai59581.2023.00044
11. Hliš, T., Četina, L., Beranič, T., and Pavlič, L., Evaluating the usability and functionality of intelligent source code completion assistants: A comprehensive review, *Appl. Sci.*, 2023, vol. 13, no. 24, p. 13061. https://doi.org/10.3390/app132413061
12. Yao, Y., Duan, J., Xu, K., Cai, Y., Sun, Z., and Zhang, Y., A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly, *High-Confidence Comput.*, 2024, vol. 4, no. 2, p. 100211. https://doi.org/10.1016/j.hcc.2024.100211
13. Horne, D., PwnPilot: Reflections on trusting trust in the age of large language models and AI code assistants, *2023 Congress in Computer Science, Computer Engineering, amp;amp; Applied Computing (CSCE)*, Las Vegas, 2023, IEEE, 2023, pp. 2457–2464. https://doi.org/10.1109/csce60160.2023.00396
14. Kaniewski, S., Holstein, D., Schmidt, F., and Heer, T., Vulnerability handling of AI-generated code—Existing solutions and open challenges, *2024 Conference on AI, Science, Engineering, and Technology (AIxSET)*, Laguna Hills, CA, 2024, IEEE, 2024, pp. 145–148. https://doi.org/10.1109/aixset62544.2024.00026
15. Tony, C., Díaz Ferreyra, N.E., Mutas, M., Dhif, S., and Scandariato, R., Prompting techniques for secure code generation: A systematic investigation, *ACM Trans. Software Eng. Methodology*, 2025, vol. 34, no. 8, p. 225. https://doi.org/10.1145/3722108
16. Kitchenham, B.A., Budgen, D., and Brereton, P., *Evidence-Based Software Engineering and Systematic Reviews*, Chapman & Hall/CRC Innovations in Software Engineering and Software Development Series, Chapman and Hall/CRC, 2015. https://doi.org/10.1201/b19467
17. Zhang, H., Babar, M.A., and Tell, P., Identifying relevant studies in software engineering, *Inf. Software Technol.*, 2011, vol. 53, no. 6, pp. 625–637. https://doi.org/10.1016/j.infsof.2010.12.010
18. Popay, J., Roberts, H., Sowden, A., Petticrew, M., Britten, N., Arai, L., Roen, K., and Rodgers, M., Developing guidance on the conduct of narrative synthesis in systematic reviews, *J. Epidemiol. Community Health*, 2005, vol. 59, suppl. 1, p. a7.
19. Wohlin, C., Guidelines for snowballing in systematic literature studies and a replication in software engineering, *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, London, 2014, New York: Association for Computing Machinery, 2014, p. 38. https://doi.org/10.1145/2601248.2601268
20. Jesse, K., Ahmed, T., Devanbu, P.T., and Morgan, E., Large language models and simple, stupid bugs, *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, Melbourne, 2023, IEEE, 2023, pp. 563–575. https://doi.org/10.1109/msr59073.2023.00082
21. Storhaug, A., Li, J., and Hu, T., Efficient avoidance of vulnerabilities in auto-completed smart contract code using vulnerability-constrained decoding, *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, Florence, 2023, IEEE, 2023, pp. 683–693. https://doi.org/10.1109/issre59848.2023.00035
22. Ernst, N.A. and Bavota, G., AI-driven development is here: Should you worry?, *IEEE Software*, 2022, vol. 39, no. 2, pp. 106–110. https://doi.org/10.1109/ms.2021.3133805
23. Tony, C., Mutas, M., Ferreyra, N.E.D., and Scandariato, R., LLMSecEval: A dataset of natural language prompts for security evaluations, *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, Melbourne, 2023, IEEE, 2023, pp. 588–592. https://doi.org/10.1109/msr59073.2023.00084
24. Yang, G., Zhou, Yu., Yang, W., Yue, T., Chen, X., and Chen, T., How important are good method names in neural code generation? A model robustness perspective, *ACM Trans. Software Eng. Methodology*, 2023, vol. 33, no. 3, p. 60. https://doi.org/10.1145/3630010

25. Barke, Sh., James, M.B., and Polikarpova, N., Grounded copilot: How programmers interact with code-generating models, *Proc. ACM Program. Lang.*, 2023, vol. 7, no. oopsla1, pp. 85–111. https://doi.org/10.1145/3586030
26. Perry, N., Srivastava, M., Kumar, D., and Boneh, D., Do users write more insecure code with AI assistants?, *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, Copenhagen, 2023, New York: Association for Computing Machinery, 2023, pp. 2785–2799. https://doi.org/10.1145/3576915.3623157
27. Seo, J., Zhang, N., and Rong, Ch., Flexible and secure code deployment in federated learning using large language models: Prompt engineering to enhance malicious code detection, *2023 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, Naples, 2023, IEEE, 2023, pp. 341–349. https://doi.org/10.1109/cloudcom59040.2023.00062
28. Ren, X., Ye, X., Zhao, D., Xing, Zh., and Yang, X., From misuse to mastery: Enhancing code generation with knowledge-driven AI chaining, *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Luxembourg, 2023, IEEE, 2023, pp. 976–987. https://doi.org/10.1109/ase56229.2023.00143
29. Liu, Yu., Le-Cong, T., Widyasari, R., Tantithamthavorn, Ch., Li, L., Le, X.-B.D., and Lo, D., Refining ChatGPT-generated code: Characterizing and mitigating code quality issues, *ACM Trans. Software Eng. Methodology*, 2024, vol. 33, no. 5, p. 116. https://doi.org/10.1145/3643674
30. Asare, O., Nagappan, M., and Asokan, N., Is GitHub's Copilot as bad as humans at introducing vulnerabilities in code?, *arXiv Preprint*, 2022. https://doi.org/10.48550/arXiv.2204.04741
31. Sandoval, G.A., Pearce, H., Nys, T., Karri, R., Dolan-Gavitt, B., and Garg, S., Lost at C: A user study on the security implications of large language model code assistants, *arXiv Preprint*, 2022. https://doi.org/10.48550/arXiv.2208.09727
32. Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.D.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., et al., Evaluating large language models trained on code, *arXiv Preprint*, 2021. https://doi.org/10.48550/arXiv.2107.03374
33. Mousavi, Z., Islam, C., Moore, K., Abuadbba, A., and Babar, M.A., An investigation into misuse of java security APIs by large language models, *arXiv Preprint*, 2024. https://doi.org/10.48550/arXiv.2404.03823
34. Elgedawy, R., Sadik, J., Dutta, S., Gautam, A., Georgiou, K., Gholamrezae, F., Ji, F., Lim, K., Liu, Q., and Ruoti, S., Ocassionally secure: A comparative analysis of code generation assistants, *arXiv Preprint*, 2024. https://doi.org/10.48550/arXiv.2402.00689
35. Liu, Z., Tang, Y., Luo, X., Zhou, Y., and Zhang, L.F., No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT, *arXiv Preprint*, 2023. https://doi.org/10.48550/arXiv.2308.04838
36. Fu, Y., Liang, P., Tahir, A., Li, Z., Shahin, M., Yu, J., and Chen, J., Security weaknesses of copilot generated code in GitHub, *arXiv Preprint*, 2023. https://doi.org/10.48550/arXiv.2310.02059
37. Asare, O., Nagappan, M., and Asokan, N., A user-centered security evaluation of copilot, *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, Lisbon, 2024, New York: Association for Computing Machinery, 2024, pp. 1–11. https://doi.org/10.1145/3597503.3639154
38. Zhong, L. and Wang, Z., Can ChatGPT replace StackOverflow? A study on robustness and reliability of large language model code generation, *arXiv Preprint*, 2023. https://doi.org/10.48550/arXiv.2308.10335
39. Khoury, R., Avila, A.R., Brunelle, J., and Camara, B.M., How secure is code generated by ChatGPT?, *arXiv Preprint*, 2023. https://doi.org/10.48550/arXiv.2304.09655
40. Hamer, S., D'Amorim, M., and Williams, L., Just another copy and paste? Comparing the security vulnerabilities of ChatGPT generated code and StackOverflow answers, *2024 IEEE Security and Privacy Workshops (SPW)*, San Francisco, CA, 2024, IEEE, 2024, pp. 87–94. https://doi.org/10.1109/spw63631.2024.00014
41. Jakkal, V., Introducing Microsoft Security Copilot: Empowering defenders at the speed of AI, The Official Microsoft Blog, 2023. https://blogs.microsoft.com/blog/2023/03/28/introducing-microsoft-security-copilot-empowering-defenders-at-the-speed-of-ai/.
42. Mohsin, A., Janicke, H., Wood, A., Sarker, I.H., Maglaras, L., and Janjua, N., Can we trust large language models generated code? A framework for in-context learning, security patterns, and code evaluations across diverse LLMs, *arXiv Preprint*, 2024. https://doi.org/10.48550/arXiv.2406.12513
43. Siddiq, M.L., Roney, L., Zhang, J., and Da Silva Santos, J.C., Quality assessment of ChatGPT generated code and their use by developers, *Proceedings of the 21st International Conference on Mining Software Repositories*, Lisbon, 2024, New York: Association for Computing Machinery, 2024, pp. 152–156. https://doi.org/10.1145/3643991.3645071
44. Yu, X., Liu, L., Hu, X., Keung, J.W., Liu, J., and Xia, X., Fight fire with fire: How much can we trust ChatGPT on source code-related tasks?, *IEEE Trans. Software Eng.*, 2024, vol. 50, no. 12, pp. 3435–3453. https://doi.org/10.1109/tse.2024.3492204
45. Rabbi, Md.F., Champa, A.I., Zibran, M.F., and Islam, M.R., AI writes, we analyze: The ChatGPT Python code saga, *Proceedings of the 21st International Conference on Mining Software Repositories*, Lisbon, 2024, New York: Association for Computing Machinery, 2024, pp. 177–181. https://doi.org/10.1145/3643991.3645076
46. Majdinasab, V., Bishop, M.J., Rasheed, Sh., Moradidakhel, A., Tahir, A., and Khomh, F., Assessing the security of GitHub Copilot's generated code—A targeted replication study, *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Rovaniemi, Finland, 2024, IEEE, 2024, pp. 435–444. https://doi.org/10.1109/saner60148.2024.00051
47. Bruni, M., Gabrielli, F., Ghafari, M., and Kropp, M., Benchmarking prompt engineering techniques for secure

code generation with GPT models, *arXiv Preprint*, 2025. https://doi.org/10.48550/arXiv.2502.06039

48. Peng, J., Cui, L., Huang, K., Yang, J., and Ray, B., Cweval: Outcome-driven evaluation on functionality and security of LLM code generation, *arXiv Preprint*, 2025. https://doi.org/10.48550/arXiv.2501.08200
49. Kharma, M., Choi, S., Alkhanafseh, M., and Mohaisen, D., Security and quality in LLM-generated code: A multi-language, multi-model analysis, *arXiv Preprint*, 2025. https://doi.org/10.48550/arXiv.2502.01853
50. Sajadi, A., Le, B., Nguyen, A., Damevski, K., and Chatterjee, P., Do LLMs consider security? an empirical study on responses to programming questions, *arXiv Preprint*, 2025. https://doi.org/10.48550/arXiv.2502.14202
51. Licorish, S.A., Bajpai, A., Arora, C., Wang, F., and Tantithamthavorn, K., Comparing human and LLM generated code: The jury is still out!, *arXiv Preprint*, 2025. https://doi.org/10.48550/arXiv.2501.16857
52. Dunne, M., Schram, K., and Fischmeister, S., Weaknesses in LLM-generated code for embedded systems networking, *2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS)*, Cambridge, 2024, IEEE, 2024, pp. 250–261. https://doi.org/10.1109/QRS62785.2024.00033
53. Ambati, S.H., Ridley, N., Branca, E., and Stakhanova, N., Navigating (in)security of AI-generated code, *2024 IEEE International Conference on Cyber Security and Resilience (CSR)*, London, 2024, IEEE, 2024, pp. 1–8. https://doi.org/10.1109/csr61664.2024.10679468
54. Wang, B., Li, H., Liu, A., Yang, B., Yang, A., Zhong, Yi., Huang, W., Huang, R., Zeng, W., and Zhang, Ya., RefleXGen: The unexamined code is not worth using, *ICASSP 2025—2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Hyderabad, India, 2025, IEEE, 2025, pp. 1–5. https://doi.org/10.1109/icassp49660.2025.10890824
55. Hajipour, H., Hassler, K., Holz, T., Schönherr, L., and Fritz, M., Codelmsec benchmark: Systematically evaluating and finding security vulnerabilities in black-box code language models, *2024 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, Toronto, 2024, IEEE, 2024, pp. 684–709. https://doi.org/10.1109/SaTML59370.2024.00040
56. Black, G.S., Rimal, B.P., and Vaidyan, V.M., Balancing security and correctness in code generation: An empirical study on commercial large language models, *IEEE Trans. Emerging Top. Comput. Intell.*, 2025, vol. 9, no. 1, pp. 419–430. https://doi.org/10.1109/tetci.2024.3446695
57. Li, J., Sangalay, A., Cheng, Ch., Tian, Yu., and Yang, J., Fine tuning large language model for secure code generation, *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering*, Lisbon, 2024, New York: Association for Computing Machinery, 2024, pp. 86–90. https://doi.org/10.1145/3650105.3652299
58. Nazzal, M., Khalil, I., Khreishah, A., and Phan, N., PromSec: Prompt optimization for secure generation of functional source code with large language models (LLMs), *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, Salt Lake City, UT, 2024, New York: Association for Computing Machinery, 2024, pp. 2266–2280. https://doi.org/10.1145/3658644.3690298
59. Jamdade, M. and Liu, Yi., A pilot study on secure code generation with ChatGPT for web applications, *Proceedings of the 2024 ACM Southeast Conference on ZZZ*, Marietta, GA, 2024, New York: Association for Computing Machinery, 2024, pp. 229–234. https://doi.org/10.1145/3603287.3651194
60. Chavan, S.B. and Mondal, Sh., Do large language models recognize Python identifier swaps in their generated code?, *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, Porto de Galinhas, Brazil, 2024, New York: Association for Computing Machinery, 2024, pp. 663–664. https://doi.org/10.1145/3663529.3663869
61. Abdali, S., Anarfi, R., Barberan, C., and He, J., Securing large language models: Threats, vulnerabilities and responsible practices, *arXiv Preprint*, 2024. https://doi.org/10.48550/arXiv.2403.12503
62. Karampatsis, R.-M. and Sutton, Ch., How often do single-statement bugs occur?, *Proceedings of the 17th International Conference on Mining Software Repositories*, Seoul, 2020, New York: Association for Computing Machinery, 2020, pp. 573–577. https://doi.org/10.1145/3379597.3387491
63. Shekhar, S., Dubey, T., Mukherjee, K., Saxena, A., Tyagi, A., and Kotla, N., Towards optimizing the costs of LLM usage, *arXiv Preprint*, 2024. https://doi.org/10.48550/arXiv.2407.02395
64. Wang, J., Luo, X., Cao, L., He, H., Huang, H., Xie, J., Jatowt, A., and Cai, Y., Is your ai-generated code really safe? evaluating large language models on secure code generation with codeseceval, *arXiv Preprint*, 2024. https://doi.org/10.48550/arXiv.2407.02395
65. Stack Overflow developer survey 2023—Survey. https://survey.stackoverflow.co/2023/. Cited May 6, 2024.
66. Fernandez-Y Fernandez, C., Sánchez-Soto, E., Cisnero, J.A., and Juárez-Ramírez, R., Exploring the frontier of software engineering education with chatbots, *Program. Comput. Software*, 2024, vol. 50, no. 8, pp. 796–815. https://doi.org/10.1134/S0361768824700774
67. Moguel-Sánchez, R., Martínez-Palacios, C.S., Ocharán-Hernández, J.O., Limón, X., and Sánchez-García, A., Bots in software development: A systematic literature review and thematic analysis, *Program. Comput. Software*, 2023, vol. 49, no. 8, pp. 712–734. https://doi.org/10.1134/S0361768823080145
68. Xu, Z., Jain, S., and Kankanhalli, M., Hallucination is inevitable: An innate limitation of large language models, *arXiv Preprint*, 2024. https://doi.org/10.48550/arXiv.2401.11817

Publisher's Note. Pleiades Publishing remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.
AI tools may have been used in the translation or editing of this article.